ASSIGNMENT REPORT

Threads, Synchronization, Deadlock & CPU Scheduling

xv6 operating-system kernel extensions

Tashya Aryan • 2401MC03 • Operating Systems Lab

REPORT STATUS This report is based on the assignment specification and the project README available for preparation. Actual QEMU logs and measured completion-time data were not supplied, so runtime results are identified as expected or pending verification rather than claimed as observed.

Objective and design overview

The assignment extends xv6 with shared-address-space threads, kernel semaphores, user-space deadlock analysis, and priority-based preemptive scheduling with aging. The project README identifies xv6-RISC-V as the intended source tree. These extensions touch both kernel lifecycle code and user-level tests, so syscall declarations, dispatch entries, user stubs, and build lists must remain consistent.

Q1 — Thread creation and joining

The clone(fcn, arg, stack) interface creates a schedulable execution context that shares the caller's address space. Each thread still needs its own kernel stack, saved context/trap frame, process-table entry, and user stack. The initial user context must be arranged so execution begins at fcn(arg), with the argument in the expected register and the user stack aligned for the target ABI. join() waits for a completed child thread and reclaims thread-specific resources.

The central lifecycle decision is to separate a thread's execution resources from the shared address space. exit() must not free a page table while another thread still uses it; join() must reclaim only the finished thread's private resources. fork() and wait() must retain well-defined process semantics in the presence of threads. These rules reduce double-free and use-after-free risks.

Q2 — Counting semaphores and critical sections

A fixed-size kernel semaphore table (for example, ten entries as suggested by the assignment) stores each semaphore's count and synchronization lock. sem_init(id, value) initializes an entry; sem_wait(id) acquires a permit or sleeps while unavailable; sem_signal(id) releases a permit and wakes a waiter where required. The count and sleep/wakeup transition must be protected consistently to avoid lost wakeups.

In synctest.c, a mutex semaphore surrounds counter updates by three threads performing 100,000 increments each. The expected protected result is 300,000. The same semaphore API can implement Producer–Consumer with a five-slot buffer: empty starts at 5, full at 0, and mutex at 1. The producer waits for an empty slot before inserting; the consumer waits for a full slot before removing; both protect buffer indices and contents with mutex.

Case	Expected counter behavior	Reason
Q1: no lock	Often below 300,000	Concurrent read-modify-write increments can be lost.
Q2: mutex semaphore	Exactly 300,000	Only one thread updates the counter inside the critical section at a time.

Q3 — Deadlock detection and avoidance

The deadlock program models a resource-allocation system in user space, using static Allocation, Request/Need, and Available arrays. For detection, Work is initialized from Available. A process whose outstanding Request is no greater than Work can finish in the simulation; its allocated resources are returned to Work. Repeating this process identifies a deadlock when processes remain unfinished. Banker's Algorithm uses Need = Max - Allocation and grants a hypothetical request only if the post-request state still has a safe completion sequence.

Test scenarios and analytical outcomes

The following compact matrices illustrate the three required categories. They are analytically evaluated examples, not claimed QEMU observations; the program's printed output should be checked against them.

Scenario	Allocation / Request	Available	Expected classification
Deadlocked	Allocation: P0[1,0], P1[0,1] Request: P0[0,1], P1[1,0]	[0,0]	Neither request can be met; P0 and P1 remain unfinished.
Safe / no deadlock	Same Allocation and Request as above	[1,1]	P0 can finish and release [1,0]; then P1 can finish.
Unsafe after request	Allocation: P0[1,0], P1[1,1], P2[0,1] Max: P0[2,2], P1[2,2], P2[1,2]	[1,1] initially	Initial state is safe. Granting P0 request [1,0] leaves Available [0,1], with no process able to satisfy its remaining Need; reject the request.

The assignment also calls for a circular-wait demonstration using xv6 processes and pipes (or shared memory through clone where supported). This is distinct from the matrix simulator: a test should establish the blocking/waiting scenario and show that the detector classifies the modeled circular wait. The implementation must not deadlock the whole test environment permanently; use controlled synchronization and a safe termination strategy.

Key implementation challenges and resolutions

- Shared memory lifetime: track whether an execution context is a thread and ensure page-table ownership is not released until no surviving thread needs it.
- Semaphore correctness: update the count under a lock and use xv6 sleep/wakeup with the lock protocol required by the chosen kernel version, preventing lost wakeups and inconsistent counts.
- Deadlock versus unsafe state: deadlock detection reasons about outstanding requests of current allocations; Banker's Algorithm reasons about maximum future needs. Keep Request and Need semantics explicit in code and printed output.
- Build integration: register each syscall consistently in kernel dispatch, syscall number definitions, user declarations, and assembly/perl stubs; add each test program to the correct Makefile list.

Q4 — Priority preemptive scheduling with aging

Each process receives an integer priority in the range 1–20, where 1 is highest, 20 is lowest, and 10 is the default. set_priority(pid, priority) changes a target process's priority after validating the process ID and range; get_priority(pid) returns the current value. The scheduler selects a RUNNABLE process with the lowest numerical priority. Equal-priority choices should preserve round-robin fairness rather than repeatedly favoring the same table entry.

A per-process wait-tick counter records time spent RUNNABLE without being scheduled. Once the wait exceeds 30 ticks, aging improves the priority by decrementing the numeric value by one and resets the counter. Priority must be clamped at 1. priotest.c should fork four CPU-bound processes with distinct priorities, print their IDs and completion order, and expose wait/priority changes for the low-priority process. Because exact completion time depends on workload, timer behavior, and emulator conditions, it must be measured from the actual run rather than inferred from the priority number alone.

Scheduling observation table

Fill the completion-time column from the actual priotest output (ticks or another consistently defined elapsed-time measure). The present materials did not include execution logs, so numerical measurements are intentionally not invented.

Process	Initial priority	Expected scheduling tendency	Measured completion time
P1	1	Favored earliest, subject to workload and preemption	Not supplied
P2	5	Favored over default/low-priority work	Not supplied
P3	10	Default priority	Not supplied
P4	20	Initially least favored; aging should improve its priority	Not supplied

INTERPRETATION A successful run should show priority influencing CPU service while aging prevents indefinite starvation. Completion order need not be perfectly monotonic if workloads finish at different times; report measured ticks and the observed priority changes from the same run.

Verification status and conclusion

The available preparation materials include the assignment specification, a README, and screenshots showing source filenames and a Makefile. They do not include successful build logs, QEMU output for the four tests, genuine test screenshots, or measured scheduling times. Therefore this report documents the design and analytical test expectations, but does not assert that the implementation has passed runtime verification.

Before final submission, build the complete xv6 tree, boot it in QEMU, run threadtest, synctest, deadlock, and priotest, compare the outputs with the expectations above, and capture one genuine terminal screenshot per question. Replace the pending completion-time entries with measured values and update any implementation-problem descriptions to match issues actually encountered and resolved.